

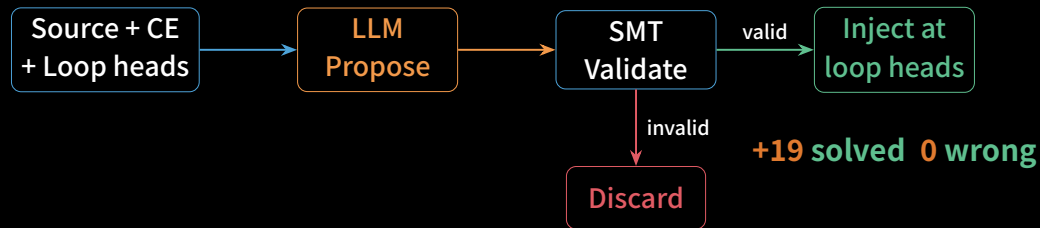
VGuide: LLM-Guided Predicate Refinement in CPAChecker

Vocabulary-Guided CEGAR for SV-COMP Reachability & Overflow

黃思維 R14K41044

2026-06-16

This Work in One Slide



LLM proposes; SMT certifies; CEGAR converges.

The Bottleneck

```
count_up_down-1
```

```
int x = 0, y = n;  
while (x < n) {  
    x++; y--;  
}  
assert(y == 0);
```

Refinements	69
Wall time	75 s
Result	UNKNOWN

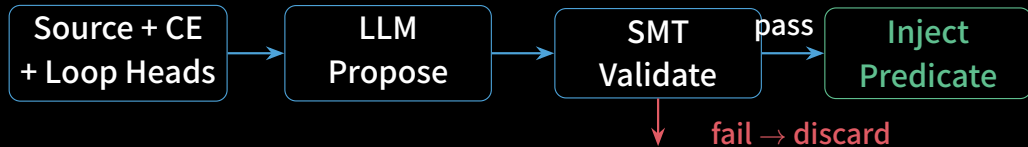
Interpolation sees one counterexample
at a time — cannot discover
cross-iteration relations:

$$y = n - x \quad (\text{the loop invariant})$$
$$x \geq 0, y \leq n$$

After 69 rounds, budget exhausted.

The Idea: LLM as Predicate Oracle

LLM guesses predicates — CPAChecker decides.



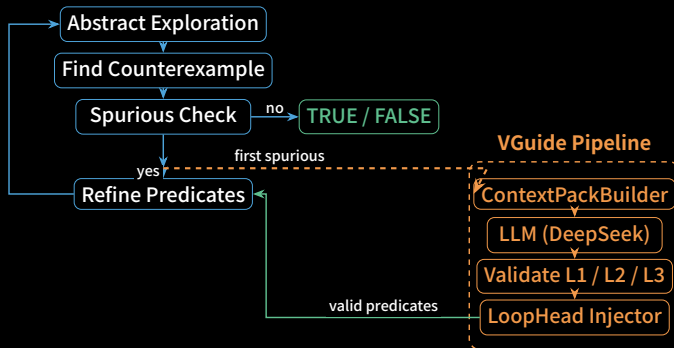
Ask for invariants,
not full proofs

Validate with SMT,
not heuristics

Inject only if
formally valid

No valid predicate $\Rightarrow$ no injection. Soundness preserved.

VGuide: Full System Architecture



Design: thinking Disabled

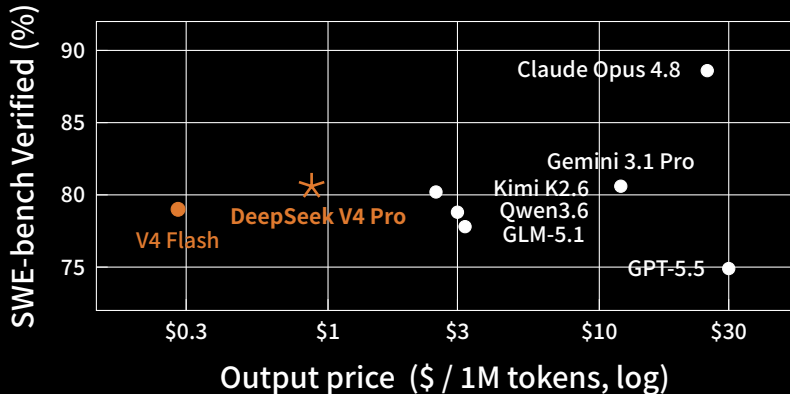
More rounds within 300 s

⇒ better PAR-2 than one
deep call

Predicates are short C
expressions
— no chain-of-thought needed.
Fast call also cuts cost per task.

Mode	Latency	PAR-2 avg
Thinking ON	52–88 s	225–258 s
Thinking OFF	2.4 s	228 s

Why DeepSeek V4 Pro?



Predicates are C expressions $\Rightarrow$ **code reasoning**; **V4 Pro on the value frontier**.

Prompt Design: What VGuide Sends the LLM

Input (V_0 call)

1. **C source** — boilerplate stripped
2. **Assertion** — `assert(y == 0)`
3. **CFA loop head labels** — **only** valid JSON keys

Output

SMT-LIB2 prefix, JSON:

```
{"N19": [  
  "(= y (- n x))",  
  "(>= x 0)",  
  "< x n)"  
]}
```

✗ invalid → discarded

✓ valid → injected at N-label

Results: Loops ReachSafety (764 tasks @ 300 s)

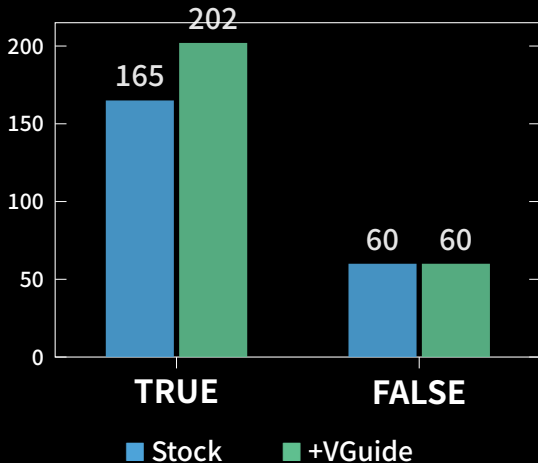
Predicate-only baseline

	Stock	+VGuide
Solved	225	262
PAR-2 avg	426.2 s	399.7 s
Δ	+37 solved -26.5 s	

Portfolio (svcomp26)

	Stock	+VGuide
Solved	486	493
PAR-2 avg	222.5 s	216.9 s
Δ	+7 solved -5.6 s	

Where the Gains Land: **TRUE** Only



TRUE **+37 / +7**

FALSE **+0 / +0**

predicate-only / portfolio

Hints help **proofs**,
not **bug search**.

Main Results @ 300 s (Competition Grade)

Category	Stock	VGuide	Net gain	PAR-2
Reachability	479	494	+15	−11.3 s
NoOverflow	362	366	+4	−5.4 s
Total			+19	

Win median: Reach 7.0 s Overflow 4.4 s

Case Study: count_up_down-1

The loop

```
int x=0, y=n;  
while (x < n) {  
    x++; y--;  
}
```

```
assert(y == 0);
```

Missing: $y = n - x, x \geq 0$

	Stock	VGuide
Ref.	69	2
Time	75.4 s	3.9 s
Result	UNKNOWN	TRUE ✓

LLM proposes the cross-iteration relation.

Proof converges in 2 refinements.

Failure Case: nested-3

Regression

	Ref.	Wall
stock (TRUE)	2	1.3 s
vguide (UNK)	42	55.9 s

Cause

LLM injects both

`st=0` and `st=1`

at the same loop head

```
int a=0,b=0,c=0,st=0;
while(1) {
    st=1;
    for(c=0; c<200000; c++)
        if (c==last) st=0;
    if(st==0 && c==last+1)
        {a+=3; b+=3;}
    else {a+=2; b+=2;}
    assert(a==b);
}
```

Conclusion

1. LLM is a **predicate oracle** — not a verifier.
2. SMT / CPAChecker **preserves soundness**.
3. **+19** tasks solved @ 300 s.

Q & A

VGuide: LLM-Guided Predicate Refinement in CPAchecker